

SYSTEM DOCUMENTATION

TOMCL COLD CHAIN

System Knowledge & Concepts

Architecture · Data flow · Database design · Python modules · Flask UI · Security

15

SECTIONS

3

SYSTEM LAYERS

12

UI TEMPLATES

2s

POLL CYCLE

Temperature · Humidity · Door status · Defrost cycles · Batch inventory

Aligned with the Tomcl Package codebase structure · Internal reference document

0

Contents

What is inside this document

- 1 What This System Does**
Scope, tracked signals and the three layers
- 2 High-Level Architecture**
Components and how they connect
- 3 Project Structure**
Folders and files at a glance
- 4 Core Concepts**
Chiller rooms, subtables, UI modes
- 5 Data Flow (End to End)**
From room creation to live charts
- 6 Database Design**
Tables, columns, relationships, concurrency
- 7 Python Modules Explained**
Fetcher, DB layer, Flask app, store, auth
- 8 Flask Dashboard (UI)**
Stack, app shell, routes, signature card
- 9 Authentication & Roles**
Operational vs admin, sessions
- 10 Change-Only Write Logic**
Two layers of deduplication
- 11 Live Data vs Mock Data**
What is real and what is demo
- 12 Utility Scripts**
Repair and password recovery
- 13 How to Run Everything**
Setup and first-time flow
- 14 Common Issues & Fixes**
Troubleshooting matrix
- 15 Glossary**
Terms used across the system

Mental model in one line

An admin registers chiller rooms in SQLite → a background fetcher writes change-only readings → the Flask dashboard reads those tables for live cards and charts.

1

What This System Does

Monitoring refrigerated storage for a meat cold chain

TOMCL Cold Chain monitors refrigerated storage units (“chillers”) for a meat cold-chain operation. It continuously tracks the environmental signals that determine whether stored cargo stays safe.

Temperature (°C) Per-chiller readings stored as change-only history.	Humidity (%) Relative humidity captured alongside temperature.	Door status Open / closed events, open duration and alarms.
Defrost cycles Defrost flag taken from the API raw snapshot.	Batch inventory Cargo stored per chiller (demo, in-memory).	Admin configuration Rooms, sensors, users and thresholds.

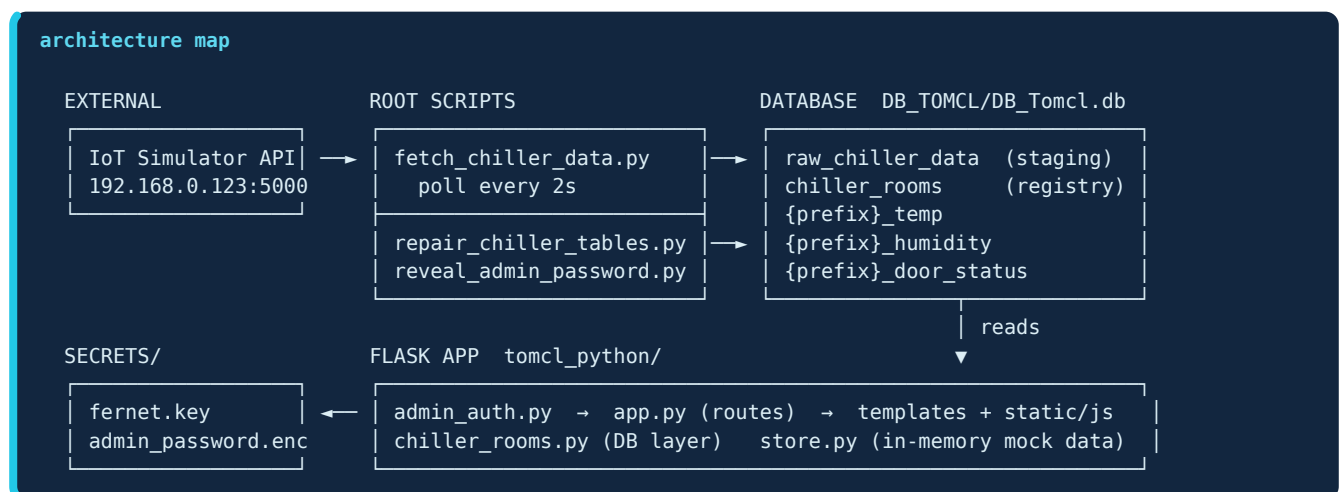
The three main layers

Layer	Purpose
Data fetcher	Polls an IoT simulator/API and writes readings into SQLite
SQLite database	Single source of truth for telemetry and the chiller room registry
Flask dashboard	Web UI for operators and admins to view and manage the system

2

High-Level Architecture

Components and how they connect



Key idea

Everything persistent lives in **one SQLite file** (DB_Tomcl.db). The fetcher and the Flask app both talk to it through `chiller_rooms.py`. Operational demo data — batches, users, alerts — lives in memory via `store.py`.

3

Project Structure

Folders and files at a glance

directory tree

```

Tomcl Package/
├── fetch_chiller_data.py      # Background poller: API → raw → subtables
├── repair_chiller_tables.py   # Creates missing subtables for existing rooms
├── reveal_admin_password.py   # Decrypts admin password (recovery only)
├── DB_TOMCL/
│   ├── DB_Tomcl.db           # Single SQLite database (telemetry + registry)
│   ├── DB_Tomcl.sqbpro       # DB Browser project file
│   └── tomcl_write.lock       # Runtime lock when admin is writing (optional)
├── secrets/
│   ├── fernet.key            # Outside application code (security)
│   ├── admin_password.enc     # Encryption key for admin password
│   └── PASSWORD_RECOVERY.txt  # Encrypted admin password blob
│                               # Human recovery instructions
├── tomcl_python/
│   ├── app.py                # Flask web dashboard
│   ├── store.py              # Routes, sessions, role switching
│   ├── chiller_rooms.py      # In-memory data (batches, users, alerts, settings)
│   ├── admin_auth.py         # SQLite: rooms, subtables, telemetry queries
│   ├── requirements.txt       # Admin password gate (@require_admin)
│   ├── static/
│   │   ├── css/custom.css    # flask, cryptography
│   │   ├── css/custom.css    # Chiller card styles, splash, animations
│   │   ├── js/ui.js          # Sidebar, filters, Chart.js, modals
│   │   └── img/TOMCL-Logo.jpeg
│   └── templates/
│       ├── base.html         # Jinja2 HTML pages (12 templates)
│       ├── overview.html     # App shell (sidebar, navbar, splash)
│       ├── detail.html       # Chiller device cards grid
│       ├── batches.html      # Analytics + Chart.js telemetry
│       ├── alerts.html       # Inventory & dispatch
│       ├── settings.html     # Static alert log table
│       ├── admin_login.html  # Threshold configuration
│       ├── admin_overview.html # Admin password gate
│       ├── chiller_rooms.html # Admin KPI dashboard
│       ├── device_config.html # Create/delete chiller rooms
│       ├── users.html        # IoT gateways, sensors, calibration
│       └── audit_logs.html   # User CRUD (mock)
│                               # Security audit (mock)

```

4

Core Concepts

The vocabulary the whole system is built on

4.1 · Chiller room

A **chiller room** is a registered cold-storage unit (e.g. "Chiller 1"). When an admin creates one, a row is inserted into **chiller_rooms** and three subtables are created automatically. The table prefix is derived from the name: "Chiller 1" → Chiller_1.

{prefix}_temp Temperature history — one row per change.	{prefix}_humidity Humidity history — one row per change.	{prefix}_door_status Door events: unlock row, then lock + duration.
---	--	---

4.2 · Raw vs subtables

Table	Role
raw_chiller_data	Staging area — every changed API reading as a full snapshot row
{prefix}_temp	Normalized temperature history (one row per change)
{prefix}_humidity	Normalized humidity history (one row per change)
{prefix}_door_status	Door events (unlock row → lock row + duration)

The fetcher always tries to **distribute from raw into subtables**, even when the API is offline.

4.3 · Single database rule

One file, one truth

All chiller data must live in DB_TOMCL/DB_Tomcl.db. There is no separate registry database in production — the legacy chiller_registry.db was migrated and removed.

4.4 · Two UI modes

Operational Manager Day-to-day monitoring: overview, analytics, batches, alerts. No password.	System Admin Configuration: rooms, users, IoT, audit. Password required.
---	--

4.5 · Fetcher independence

fetch_chiller_data.py runs **outside** tomcl_python/. It imports chiller_rooms from that folder but is started as its own process, keeping polling separate from the web server.

5

Data Flow (End to End)

From room creation to a rendered chart

Step 1 — Admin creates a chiller room

```
Browser → POST /device-config/create-room (or /chiller-rooms)
        → register_chiller_room("Chiller 1")
        → INSERT chiller_rooms
        → CREATE Chiller_1_temp, Chiller_1_humidity, Chiller_1_door_status
```

Step 2 — Fetcher polls (every 2 seconds)

#	Inside poll_once()
1	Skip if the admin write lock is held (tomcl_write.lock)
2	Optional live fetch: GET http://192.168.0.123:5000/api/dashboard — fallback /api/readings?limit=1
3	Compare the new reading with the last raw row for that chiller → insert only if changed
4	Always distribute: replay matching raw_chiller_data rows through apply_reading_to_room()

Step 3 — Dashboard reads live metrics

```
/overview → list_chiller_rooms() → latest_live_metrics(prefix)
          → temp, humidity, doorOpen on the chiller cards

/detail   → telemetry_from_subtables(chiller, date, range)
          → Chart.js line chart + door event logs
```

Offline behaviour

If the API at 192.168.0.123:5000 is down, the fetcher logs *API unreachable* and continues. Distribution still runs from existing raw rows, and the dashboard still shows data already in subtables.

6

Database Design

Registry, staging and per-chiller subtables

6.1 · Registry — `chiller_rooms`

Column	Type	Description
<code>id</code>	INTEGER PK	Auto-increment room ID
<code>name</code>	TEXT UNIQUE	Display name, e.g. "Chiller 1"
<code>table_prefix</code>	TEXT UNIQUE	Subtable prefix, e.g. "Chiller_1"
<code>created_at</code>	TEXT	ISO timestamp

6.2 · Staging — `raw_chiller_data`

Stores full API snapshots. Key columns:

Column	Description
<code>chiller_id</code>	Links to the room name (e.g. "Chiller 1")
<code>timestamp</code>	Reading time
<code>temperature_c</code>	Temperature
<code>humidity_percent</code>	Humidity
<code>door_status</code>	unlocked / locked / open / closed
<code>defrost_active</code>	Defrost flag from the API
<code>chiller_status</code>	General status string

6.3 · Per-chiller subtables

<code>{prefix}_temp</code>	Description	<code>{prefix}_humidity</code>	Description
<code>Date</code>	Date portion	<code>Date</code>	Date portion
<code>time_stamp</code>	Time portion	<code>time_stamp</code>	Time portion
<code>temp</code>	Temperature value	<code>humidity</code>	Humidity value

<code>{prefix}_door_status</code>	Description
<code>Date</code>	Event date
<code>time_stamp_unlocked</code>	When the door opened
<code>time_stamp_locked</code>	When the door closed (NULL while open)
<code>duration</code>	How long the door was open

6.4 · Relationships (logical)

```
chiller_rooms (1) → (N) raw_chiller_data      [match by chiller_id / name / prefix]
chiller_rooms (1) → (1) {prefix}_temp
chiller_rooms (1) → (1) {prefix}_humidity
chiller_rooms (1) → (1) {prefix}_door_status
```

SQLite does not enforce foreign keys; matching is by naming convention and string IDs.

6.5 · Concurrency settings

WAL journal mode Allows concurrent reads while a write is in progress.	busy_timeout Retries automatically when the database is locked.
tomcl_write.lock File lock so the fetcher pauses during admin writes.	Close DB Browser Do this before admin create/delete to avoid long locks.

7

Python Modules Explained

What each file is responsible for

7.1 · `fetch_chiller_data.py` (root) — background data pipeline

Function	What it does
<code>fetch_snapshot()</code>	GET JSON from the simulator API
<code>normalize_row()</code>	Maps API fields to DB column names
<code>raw_changed()</code>	Compares all tracked fields; skips duplicate raw inserts
<code>try_fetch_and_append_raw()</code>	Live path: API → <code>raw_chiller_data</code>
<code>distribute_from_existing_raw()</code>	Offline path: raw → subtables for every room
<code>poll_once()</code>	One cycle: optional fetch + always distribute
<code>main()</code>	Loop every <code>POLL_SECONDS</code> (2s)

constants

```
SOURCE_BASE = "http://192.168.0.123:5000"
DB_PATH      = DB_TOMCL/DB_Tomcl.db
```

7.2 · `tomcl_python/chiller_rooms.py` — the brain of persistence

Function	What it does
<code>register_chiller_room(name)</code>	Create the registry row + 3 subtables
<code>delete_chiller_room(id)</code>	Drop subtables + delete the registry row
<code>list_chiller_rooms()</code>	Return all registered rooms (cached ~1.5s)
<code>apply_reading_to_room(conn, prefix, reading)</code>	Change-only write to temp / humidity / door tables
<code>latest_live_metrics(prefix)</code>	Latest temp, humidity, door-open flag for overview cards
<code>telemetry_from_subtables(chiller, date, range)</code>	Chart data for 24h / 7 Days / 30 Days
<code>connect()</code>	SQLite connection with WAL + busy timeout
<code>acquire_write_lock() / release_write_lock()</code>	Admin write coordination

Name slugging: "Chiller 1" → `Chiller_1` (spaces become underscores).

7.3 · tomcl_python/app.py — Flask web server and routing

- Runs on `http://127.0.0.1:5000`
- On startup: `STORE.sync_from_rooms(list_chiller_rooms())` — syncs the in-memory chiller list from the DB
- `nav_context(active_tab)` builds the sidebar menu based on role
- Operational routes: `/overview`, `/detail`, `/batches`, `/alerts`, `/settings`
- Admin routes: `/admin`, `/users`, `/device-config`, `/chiller-rooms`, `/audit-logs` — all `@require_admin`
- Session keys: `role`, `admin_authenticated`

7.4 · tomcl_python/store.py — in-memory application state

Mirrors the original React AppContext.

Data	Persisted?
chillers	Synced from DB rooms (not written back)
batches	In-memory only
users	In-memory only
notifications	In-memory only
settings	In-memory only
ALERT_LOGS, AUDIT_LOGS, GATEWAYS	Static mock constants

Legacy note

`telemetry_for()` in this file is legacy mock chart data from the original React app. The Flask app now uses `telemetry_from_subtables()` from `chiller_rooms.py` for real charts.

7.5 · tomcl_python/admin_auth.py — secure admin access

`secrets/fernet.key` → Fernet encryption key
`secrets/admin_password.enc` → Encrypted password blob

Function	What it does
verify_admin_password(password)	Decrypt the expected password and compare
@require_admin	Decorator: redirect to <code>/admin/login</code> if not authenticated
admin_authenticated()	Check the session flags

Recovery: run `python reveal_admin_password.py` or read `secrets/PASSWORD_RECOVERY.txt`.

8

Flask Dashboard (UI)

Stack, app shell, routes and the signature card

8.1 · Technology stack

Server Flask (Python)	Templates Jinja2 HTML	Styling Tailwind CSS v4 (CDN) + custom.css
Icons Lucide (CDN)	Charts Chart.js 4 (detail page)	Font Source Sans 3

8.2 · App shell (base.html)

Shared layout for all pages except the admin login:

Element	Behaviour
Splash screen	First visit only — 2.4s, tracked in sessionStorage
Sidebar	Role switcher + navigation, open/closed state in localStorage
Navbar	Logo, notifications, user badge
Main content	Page-specific Jinja block
Toast	Flash success messages, auto-dismiss after 3.5s

Client behaviour lives in `static/js/ui.js`: sidebar open/close, overview filter pills and search, the dual-axis Chart.js chart, batch modals, user modals and multiselects.

8.3 · Pages and routes

Page	Route	Role	Data source
Dashboard Overview	/overview	Operational	Live DB subtables
Chiller Analytics	/detail/<id>	Operational	Live DB subtables
Batches & Inventory	/batches	Operational	In-memory STORE
Alerts & Reports	/alerts	Operational	Static ALERT_LOGS
System Settings	/settings	Both	In-memory STORE.settings
Admin Login	/admin/login	Gate	Fernet password
Admin Overview	/admin	Admin	DB rooms + mock KPIs
Chiller Rooms	/chiller-rooms	Admin	DB chiller_rooms
User Management	/users	Admin	In-memory users
IoT & Sensors Config	/device-config	Admin	DB + mock gateways
Audit & Security Logs	/audit-logs	Admin	Static AUDIT_LOGS

8.4 · Signature UI — the chiller device card

.chiller-device-card Card container with hover animation.	.chiller-device-halo Colored glow: blue / red alarm / amber defrost.
.chiller-device-dock Blue bottom section with door + status.	.device-switch Pill toggle showing Normal / Alarm / Defrost.

State is driven by `isAlarm` (door open), `isDefrosting`, and live temp/humidity from the database.

9

Authentication & Roles

Who can see what, and how it is enforced

9.1 Operational mode — no password

Default mode for floor staff. Full access to overview, analytics, batches, alerts and settings. Cannot create or delete chiller rooms, and cannot manage users.

9.2 Admin mode — password required

Configuration role. Unlocks rooms, users, IoT config and audit logs behind a Fernet-encrypted password stored outside the codebase.

Admin flow

```
User selects "System Admin" in the sidebar
→ POST /set-role (role=admin)
→ Redirect to /admin/login

User enters password
→ verify_admin_password() using the Fernet secrets
→ session["admin_authenticated"] = True
→ Redirect to /admin

Any @require_admin route without auth
→ Redirect to /admin/login?next=<path>
```

9.3 · Session keys

Key	Values	Meaning
role	"operational" or "admin"	Selected mode
admin_authenticated	True / absent	Password verified

Fallback rule

If role=admin but the session is not authenticated, the UI falls back to the operational menu.

10

Change-Only Write Logic

Two layers that keep the database honest

The system avoids duplicate rows when values have not changed. This happens at two layers.

Layer 1 — Raw table (fetch_chiller_data.py)

Before inserting into `raw_chiller_data`, all fields in `COMPARE_FIELDS` are compared — timestamp, `chiller_status`, `defrost_active`, `door_status`, `temperature_c`, `humidity_percent` and others. **Insert only if any field differs** from the previous row for that `chiller_id`.

Layer 2 — Subtables (apply_reading_to_room())

Metric	Rule
Temperature	INSERT into {prefix}_temp only if temperature_c ≠ the last stored temp
Humidity	INSERT into {prefix}_humidity only if humidity_percent ≠ the last stored humidity
Door unlocked	INSERT a new door row when the status changes to unlocked/open
Door locked	UPDATE the open door row: set time_stamp_locked + compute duration

Door open states unlocked, open → treated as unlocked	Door closed states locked, closed → treated as locked
--	--

Why this matters

It reduces database size, makes charts meaningful — one point per actual change — and matches how real sensors behave.

11

Live Data vs Mock Data

What is real and what is demo

Feature	Source	Persisted to DB?
Chiller room registry	SQLite <code>chiller_rooms</code>	Yes
Temperature / humidity history	SQLite subtables	Yes
Door event history	SQLite <code>{prefix}_door_status</code>	Yes
Raw API readings	SQLite <code>raw_chiller_data</code>	Yes
Overview card temp/hum/door	<code>latest_live_metrics()</code>	Live read
Analytics charts	<code>telemetry_from_subtables()</code>	Live read
Defrost on overview cards	In-memory default (False)	Not from DB yet
Defrost logs on detail page	Empty from the DB path	Not implemented
Batches & inventory	<code>STORE.batches</code>	In-memory only
Users	<code>STORE.users</code>	In-memory only
Alerts table	<code>ALERT_LOGS</code> constant	Static mock
Audit logs	<code>AUDIT_LOGS</code> constant	Static mock

Feature	Source	Persisted to DB?
IoT gateways	GATEWAYS constant	Static mock
Notifications	STORE.notifications	In-memory only
Threshold settings	STORE.settings	In-memory only

Summary

Telemetry is **real** (from SQLite). Business, HR and security demo pages are **mock** unless extended later.

12

Utility Scripts

Repair and recovery helpers

repair_chiller_tables.py

Ensures the schema exists and creates missing `_temp`, `_humidity` and `_door_status` tables for all registered rooms. Run it when subtables are missing after a room creation failed due to a DB lock.

reveal_admin_password.py

Decrypts and prints the admin password using `secrets/fernet.key` and `admin_password.enc`. Recovery only — never commit or share the output.

13

How to Run Everything

Setup, startup order and first-time flow

Prerequisites

Python 3.10+ · `pip install -r tomcl_python/requirements.txt` (Flask, cryptography)

Full stack — recommended order

Terminal 1 — Flask dashboard

```
cd "...\\Tomcl Package\\tomcl_python"
pip install -r requirements.txt
python app.py # open http://127.0.0.1:5000
```

Terminal 2 — Data fetcher (after creating at least one chiller room)

```
cd "...\\Tomcl Package"
python fetch_chiller_data.py
```

First-time setup

#	Step
1	Open the dashboard → switch to System Admin → enter the admin password
2	Go to Chiller Rooms or IoT & Sensors Config → Create Chiller Room (e.g. "Chiller 1")
3	Start <code>fetch_chiller_data.py</code>
4	View Dashboard Overview — cards show live temp/humidity from the subtables

maintenance commands — run from the Tomcl Package folder

```
python repair_chiller_tables.py # repair missing tables (close DB Browser first)
python reveal_admin_password.py # admin password recovery
```


14

Common Issues & Fixes

Troubleshooting matrix

Problem	Cause	Fix
Empty overview / no chiller cards	No rooms in chiller_rooms	Create a room in Admin
Subtables empty but raw has data	Fetcher not running or ID mismatch	Run the fetcher; ensure chiller_id in raw matches the room name
database is locked	DB Browser open or a long admin write	Close DB Browser; wait 2s; retry
Admin delete hangs	SQLite lock contention	Fixed: non-blocking delete + write lock
API unreachable in fetcher logs	Simulator off at 192.168.0.123:5000	Normal — the fetcher continues from existing raw data
Charts empty for today	No subtable rows for the selected date	Pick a date that has data in _temp / _humidity
Slow admin page load	Was calling repair on every page view	Removed auto-repair on page load

15

Glossary

Terms used across the system

Term	Meaning
Chiller room	A registered cold storage unit in the system
Table prefix	Slug used to name subtables (e.g. Chiller_1)
Raw data	Full API snapshot rows in raw_chiller_data
Subtable	Per-chiller normalized table (_temp, _humidity, _door_status)
Change-only write	Insert or update only when a value actually changes
Fetcher	fetch_chiller_data.py background poller
Operational mode	Day-to-day monitoring role (no admin password)
Admin mode	Configuration role (password required)
STORE	In-memory singleton in store.py for mock/demo data
Fernet	Symmetric encryption used for admin password storage
WAL mode	SQLite Write-Ahead Logging for better concurrent access

Mental Model — one paragraph

An admin **registers chiller rooms** in SQLite, which creates **three metric tables per chiller**. A **background fetcher** optionally pulls live readings from an IoT simulator, stores **change-only rows** in `raw_chiller_data`, and **distributes** them into `temp` / `humidity` / `door` subtables — even when the API is offline. The **Flask dashboard** reads those subtables for **live overview cards and analytics charts**, while **batches, users, alerts and audit logs** remain **in-memory demo data** for operational workflows. **Admin routes** are protected by a **Fernet-encrypted password** stored outside the codebase in `secrets/`.

Document version: aligned with the Tomcl Package codebase structure. Update this document when adding new routes, tables or data sources.